Javascript Events and its Handlers

Javascript Events

- An **event** is an action that occurs in the web browser, which the web browser feedbacks to you so that you can respond to it.
- **Example** - click event by displaying a dialog box
- Each event may have an **event handler** which is a block of code that will execute when the event occurs.
- An event handler is also known as an **event listener.**
- It listens to the event and executes when the event occurs.

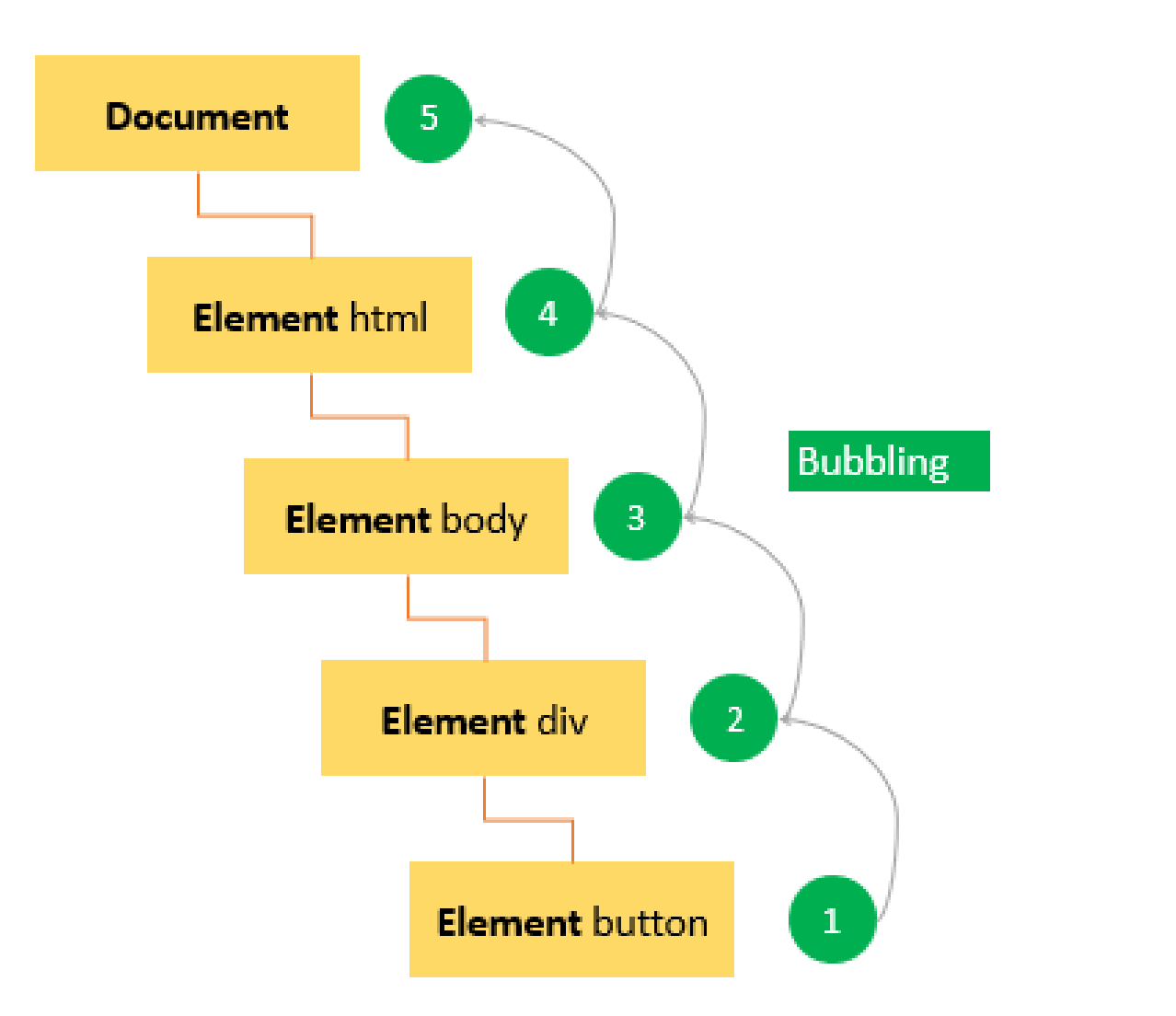
Event flow

```
<!DOCTYPE html>
<html>
<head>
    <title>JS Event Demo</title>
</head>
<body>
    <div id="container">
        <button id='btn'>Click Me!</button>
    </div>
</body>
```

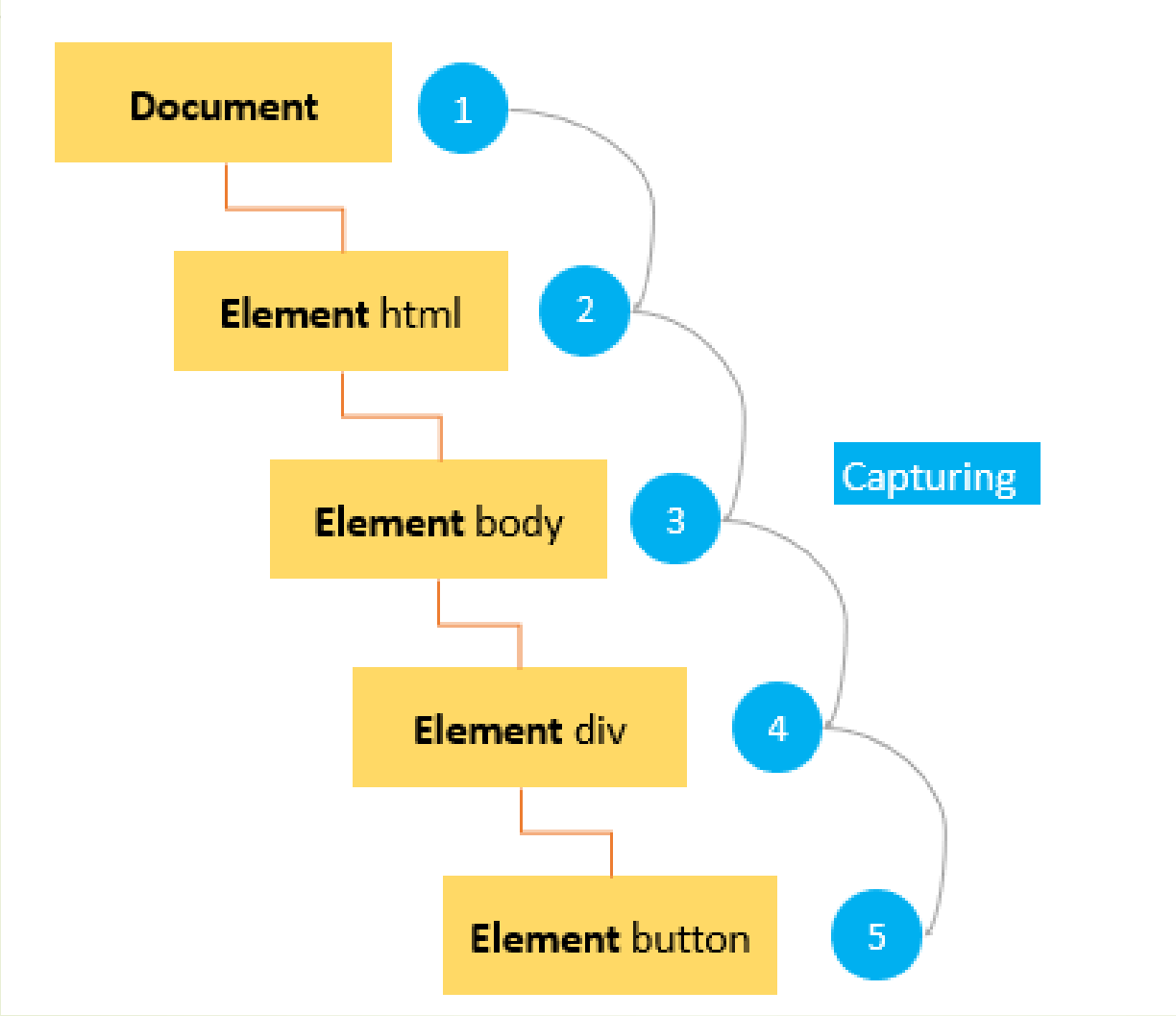
Event flow

- Event flow explains the order in which events are received on the page from the element where the event occurs and **propagated through the DOM tree.**
- There are two main event models: event bubbling and event capturing.
- In the **event bubbling model**, an event starts at the most specific element and then flows upward toward the least specific element (the document or even window).
- In the **event capturing model**, an event starts at the least specific element and flows downward toward the most specific element.

Event Bubbling Model



Event Capturing Model



Event Handlers

- An event can be handled by **one or multiple event handlers.**
- If an event has multiple event handlers, all the event handlers will be executed when the event is fired.
- There are **three ways** to assign event handlers.

✓ HTML event handler attributes

✓ DOM Level 0 event handlers

✓ DOM Level 2 event handlers

HTML Event Handler Attributes

- Event handlers typically have names that begin with on, for example, the event handler for the click event is onclick.
- To assign an event handler to an event associated with an HTML element, you can use an **HTML attribute with the name of the event handler**.

```
<input type="button" value="Save" onclick="alert('Clicked!')">
<script>
    function showAlert() {
        alert('Clicked!');
    }
</script>
<input type="button" value="Save" onclick="showAlert()">
```

DOM Level 0 event handlers

- Each element has event handler properties such as **onclick.** To assign an event handler, you set the property to a function.

```
let btn = document.querySelector('#btn');

btn.onclick = function() {
    alert('Clicked!');
};
```

DOM Level 2 event handlers

- DOM Level 2 Event Handlers provide two main methods for dealing with the registering/deregistering event listeners:
- **addEventListener()** – register an event handler
- **removeEventListener()** – remove an event handler

AddEventListener()

- The **addEventListener()** method accepts three arguments:
- an event name,
- an event handler function,
- a Boolean value that instructs the method to call the event handler during the capture phase (true) or during the bubble phase (false).

Example - AddEventListener()

```
let btn = document.querySelector('#btn');
btn.addEventListener('click',function(event) {
   alert(event.type); // click
});
```

It is possible to add multiple event handlers to handle a single event.

```
let btn = document.querySelector('#btn');
btn.addEventListener('click',function(event) {
   alert(event.type); // click
});

btn.addEventListener('click',function(event) {
   alert('Clicked!');
});
```

RemoveEventListener()

The **removeEventListener()** removes an event listener that was added via the addEventListener().

```
let btn = document.querySelector('#btn');

// add the event listener
let showAlert = function() {
    alert('Clicked!');
};
btn.addEventListener('click', showAlert);

// remove the event listener
btn.removeEventListener('click', showAlert);
```

Mouse Events

Event Performed	Event Handler	Description
click	onclick	When mouse click on an element
mouseover	onmouseover	When the cursor of the mouse comes over the element
mouseout	onmouseout	When the cursor of the mouse leaves an element
mousedown	onmousedown	When the mouse button is pressed over the element
mouseup	onmouseup	When the mouse button is released over the element
mousemove	onmousemove	When the mouse movement takes place.

Keyboard Events

Event Performed	Event Handler	Description
Keydown & Keyup	onkeydown & onkeyup	When the user press and then release the key

Form Events

Event Performed	Event Handler	Description
focus	onfocus	When the user focuses on an element
submit	onsubmit	When the user submits the form
blur	onblur	When the focus is away from a form element
change	onchange	When the user modifies or changes the value of a form element

Windows Events

Event Performed	Event Handler	Description
load	onload	When the browser finishes the loading of the page
unload	onunload	When the visitor leaves the current webpage, the browser unloads it
resize	onresize	When the visitor resizes the window of the browser

Examples – OnLoad Events

To handle the load event on the images, you can use the **addEventListener()** method of the image elements.

```
<body>
    <img id="logo">
    <script>
        let logo = document.querySelector('#logo');

        logo.addEventListener('load', (event) => {
            console.log('Logo has been loaded!');
        });

        logo.src = "logo.png"
    </script>
</body>
```

Examples – OnLoad Events

You can assign an onload event handler directly using the onload attribute of the <img> element.

```
<img id="logo"
    src="logo.png"
    onload="console.log('Logo loaded!')">
```

Examples – OnLoad Events

If you create an **image element dynamically**, you can assign an onload event handler before setting the src property.

```
window.addEventListener('load'  () => {
    let logo = document.createElement('img');
    // assign and onload event handler
    logo.addEventListener('load',  (event) => {
        console.log('The logo has been loaded');
    });
    // add logo to the document
    document.body.appendChild(logo);
    logo.src = 'logo.png';
});
```

Examples – Mouse Events

```
<body>
<script>
    function mouseoverevent()
    {
        alert("This is Javascript");
    }
</script>
<p onmouseover="mouseoverevent()"> Keep cursor over me</p>
</body>
```